ACTIVIDAD PRÁCTICA

Diseño arquitectural del sistema con IA

FerreMax Informe de arquitectura

FerreMax requiere coordinar cotizaciones, pedidos e inventario sin comprometer mercancía inexistente. La arquitectura propuesta organiza las reglas del negocio con Domain Model, una Service Layer y adaptadores de persistencia. El recorrido detallado cubre la consulta de disponibilidad, la revalidación de cotizaciones, la conversión a pedido, la confirmación con reserva y la cancelación antes del surtido.

Empresa asignada	FerreMax
Integrantes	Roberto Santana Trujillo 2227084 Isaías Leonardo Salazar Torres 2227016
Fecha de entrega	Revisión 16 de octubre de 2026 Entrega 17 de octubre de 2026 a las 23:59

Profesor Miguel Ángel Gómez Marroquín

1 Preparación del caso

FerreMax necesita que cotizaciones, pedidos e inventario compartan información coherente y que las decisiones comerciales sean trazables. La propuesta organiza el software alrededor de las reglas de la Unidad 2, con Domain Model como patrón principal, una capa de aplicación que coordina casos de uso y adaptadores de persistencia para PostgreSQL alojado en Supabase.

La operación crítica es confirmar un pedido y registrar sus reservas. Ambos cambios, junto con su auditoría y el evento pendiente de publicación, se guardan en una sola transacción. Si una partida no tiene disponibilidad suficiente, la confirmación y cualquier reserva provisional se revierten.

La base de código demuestra un subconjunto del sistema: cotizaciones, confirmación de contado, consulta de disponibilidad y cancelación. El mapa general conserva crédito, transferencias, parciales, surtido, entrega y devolución. El prototipo representa este subconjunto; los procesos posteriores se conservan en el mapa general del dominio.

Continuidad con la Unidad 2

Fuente	Uso en U3
EQ17FerreMax_BRD(1).pdf	Contexto de tres sucursales y almacén central; BN01-BN10, BR01-BR16, RNF01-RNF08, alcance y supuestos.
EQ17FerreMax_UserStories(1).pdf	US01-US20 y sus Gherkin; casos normales y rechazos; reserva atómica y competencia por última unidad.
EQ17FerreMax_Glosario(1).pdf	Lenguaje: SKU, física, comprometida, disponible, tránsito, cotización, pedido, reserva, crédito, transferencia y entrega.
EQ17FerreMax_BPMN(1).pdf	BPMN cotejado visualmente. Se registra H07: la reserva aparece antes del procesamiento del pago o la validación del crédito.

Hallazgos de continuidad

H01: US01 usa AC-US01-01 para dos escenarios distintos. La matriz U3 propone identificar el escenario «SKU inexistente» como AC-US01-02, conservando la referencia al ID original repetido. No se alteraron los PDFs.

H02: OB01 plantea reducir 80% las ventas sin mercancía; CE01 exige eliminarlas. Se propone conservar OB01 como meta de mejora frente a la línea base y tratar CE01 como aspiración; la arquitectura protege el invariante de no reservar mercancía inexistente. La redacción definitiva de los indicadores requiere revisión del equipo y del profesor.

H03: la guía menciona una matriz y un modelo de dominio de U2 que no aparecen como archivos separados entre los cuatro PDFs proporcionados. Se construyen para U3 a partir de los IDs existentes, sin fingir una matriz anterior no recibida.

H04: los criterios US18-US20 usan el prefijo CA, a diferencia de AC en las historias anteriores. Se conservan los IDs originales para no perder vínculos.

H05: no se especifica qué resultado financiero corresponde cuando Caja registra un pago autorizado y después la reserva falla. El código revierte pedido y reserva; la atención del pago ya autorizado queda pendiente de política comercial. No se simula una devolución automática.

H06: se corrigió y fortaleció el ensayo de rollback; se amplió la auditoría con cantidades, precios y condiciones antes/después.

H07: el BPMN dibuja «Reservar para pedido confirmado» antes de «Procesar pago» o «Validar crédito» y del gateway «¿Autorizado?». U3 sigue BR05 y US06/US09: autorización registrada, confirmación y reserva atómicas. Los originales de U2 se conservan sin cambios.

Alcance y casos de uso

El proceso detallado va desde la disponibilidad y la cotización hasta la confirmación del pedido y la liberación de su reserva. Todos los casos aparecen en historias de U2. Los demás procesos se representan para conservar el panorama de FerreMax y sus vínculos, pero no se implementa una aplicación completa.

ID	Caso	Actor	Entrada	Evidencia U2	Resultado
CU01	Consultar disponibilidad global	Vendedor	SKU	BN01; BR04; US01/US02	DTO por ubicación con física, comprometida, tránsito y disponible; SKU inexistente sin existencias inventadas.
CU02	Revalidar cotización vencida	Vendedor	Folio/ID y versión, partidas y precios revisados, condiciones y vigencia	BN03; BR01-03; US04	Nueva versión vigente; versión anterior conservada; no crea reserva.
CU03	Convertir cotización aceptada en pedido	Vendedor	ID y versión de cotización; aceptación del cliente	BN03/ BN04; BR01/BR03; US05	Pedido pendiente ligado al folio y versión; disponibilidad consultada nuevamente. Vencida: rechazo y solicitud de revalidación.
CU04	Confirmar pedido de contado y reservar	Caja	Pedido, referencia de pago autorizado y ubicación de surtido	BN02; BR04/BR05; RNF02; US06/US09	Pedido confirmado y reserva como una operación. Pago rechazado o stock insuficiente: no confirma y no reserva.

ID	Caso	Actor	Entrada	Evidencia U2	Resultado
CU05	Cancelar pedido antes del surtido	Vendedor con permiso de cancelación	Pedido y motivo	BN02; BR06/BR16; RNF03; US10	Pedido cancelado, reserva liberada y modificación auditada. Pedido surtido: rechazo.

Límite de la demostración

La confirmación ejecutable es de contado y usa una ubicación por pedido. Esto delimita el prototipo, no cambia las reglas del negocio sobre surtido multiorigen, crédito o parciales. Las consultas sí devuelven las cuatro ubicaciones de demostración. Los datos, referencias de pago y actores del demo son simulados.

La vigencia de siete días proviene de un acuerdo simulado del BRD y se pasa como parámetro. La definición exacta del instante de vencimiento requiere validación: el código usa un intervalo que incluye emisión y excluye vencimiento. La moneda MXN del demo no constituye una política nueva; Dinero conserva explícitamente la moneda.

2 Selección del patrón de lógica de negocio

Criterio	Transaction Script	Table Module	Domain Model	Evidencia
Cotizaciones con vigencia y versiones	Validaciones por procedimiento; riesgo de repetirlas en convertir y revalidar.	Organiza versiones como filas; invariantes por conjunto pueden quedar dispersas.	Cotizacion protege vigencia y produce otra versión sin borrar la anterior.	BR01-03; US03-05
Confirmación, inventario y concurrencia	Una transacción puede resolverlo, pero la coordinación y reglas tienden a mezclarse.	Procesamiento por tablas adecuado para lotes; debe reconstruir vínculos entre pedido y reservas.	Pedido e InventarioUbicacion protegen sus invariantes; UoW coordina su guardado.	BR04-06; RNF02; AC-US09-01/02
Estado y ciclo de vida	Cambios de estado repartidos entre scripts; exige disciplina para evitar caminos contradictorios.	Estados expresados en filas; comportamiento separado de conceptos del negocio.	Métodos confirmar y cancelar expresan transiciones permitidas y rechazos.	US06/ US10/US15; BR05/BR06
Variantes comerciales	Cada variante añade condiciones a los procedimientos existentes.	Cambia módulos centrados en datos; puede ser útil si predominan operaciones por conjunto.	Políticas de crédito y devolución pueden evolucionar con contratos propios.	BR09-11/ BR15; BN05/BN09
Costo para este equipo	Menor estructura inicial; apropiado si el caso se redujera a operaciones simples.	Costo intermedio; natural en aplicaciones organizadas alrededor de datasets.	Más clases, interfaces y revisión de límites; exige explicar responsabilidades y evitar clases sin comportamiento.	Guía U3; 16 reglas BR y excepciones documentadas
Consultas operativas	Consulta directa por procedimiento.	Buena afinidad con agregación y reportes por tablas.	Modelo de lectura independiente: no obliga a reconstruir agregados para cada consulta.	BN01/BN10; RNF01; CQRS básico

Evaluación de Transaction Script

Transaction Script puede implementar correctamente las operaciones seleccionadas. Su ventaja es la estructura inicial sencilla: consultar, verificar condiciones, actualizar y devolver un resultado. También puede usar una transacción para evitar cambios parciales. Por lo tanto, se descarta como patrón principal por su costo de mantenimiento en este caso, no porque sea incapaz de proteger el inventario.

En FerreMax, la regla «una cotización no reserva» aparece en la emisión, la revalidación y la conversión; la disponibilidad se consulta y se comprueba otra vez al confirmar; la cancelación afecta pedido y reservas. Si cada procedimiento conserva sus propias validaciones, una corrección debe repetirse en varios lugares. Puede mantenerse coherente, pero requiere controles continuos para evitar divergencia. El riesgo crece con crédito excepcional, transferencias y parciales.

Evaluación de Table Module

Table Module concentra comportamiento sobre conjuntos de registros y resulta razonable cuando la aplicación gira alrededor de datasets. Podría facilitar reportes operativos y procesamiento por ubicación. Su costo en FerreMax sería expresar el comportamiento de cotizaciones, pedidos y reservas mediante relaciones entre filas, mientras su ciclo de vida y reglas se distribuyen en módulos centrados en tablas.

La consulta global necesita una proyección eficiente, pero eso no obliga a usar Table Module como patrón de negocio. Se propone una consulta SQL de solo lectura para el DTO de disponibilidad. La separación entre un modelo de lectura y los agregados de escritura resuelve esa necesidad sin adoptar un segundo patrón principal artificial.

Evaluación y selección de Domain Model

Domain Model permite ubicar la vigencia en Cotizacion, las transiciones en Pedido y el compromiso de cantidades en InventarioUbicacion. Las operaciones públicas expresan intenciones del negocio: revalidar, convertir, confirmar, reservar, liberar y cancelar. La identidad y el estado se conservan, y el servicio de aplicación coordina objetos que ya conocen sus reglas.

La decisión se apoya especialmente en BR01-03, BR04-06 y RNF02/AC-US09-02. Estas evidencias exigen conservar versiones, distinguir compromiso de compra de compromiso de inventario y proteger una última unidad frente a dos pedidos. La claridad de responsabilidades importa más que la cantidad de clases.

El costo aceptado es una estructura mayor: objetos, contratos, adaptadores y pruebas de colaboración. Se controla concentrando el desarrollo en cinco casos y evitando capas que no resuelvan una necesidad. Domain Model no sustituye las transacciones ni los bloqueos de PostgreSQL: las reglas se verifican sobre datos recuperados de manera consistente.

3 Mapa de dominio

El mapa general identifica siete agregados. Cada raíz controla sus invariantes y conserva referencias por identidad hacia otros agregados.

Cotizacion

Raíz y contenido Cotizacion; partidas y Vigencia

Responsabilidades Revalidar; emitir versiones; validar conversión.

Invariantes Tiene folio, versión, vigencia y estado; no reserva; vencida requiere revalidación.

Eventos CotizacionRevalidada; conversión origina PedidoCreado.

Estado del código Implementado revalidar/convertir; emisión inicial es dato de demostración.

Pedido

Raíz y contenido Pedido; PartidaPedido y referencias de cotización

Responsabilidades Confirmar, cancelar, conservar procedencia y aceptación.

Invariantes Reserva ligada a pedido confirmado; cancelar antes del surtido libera; parcial requiere aceptación.

Eventos PedidoCreado, PedidoConfirmado, PedidoCancelado.

Estado del código Contado y cancelación implementados; parciales y reducción modelados para extensión.

InventarioUbicacion

Raíz y contenido InventarioUbicacion; reservas por pedido

Responsabilidades Reservar, liberar y calcular disponible por SKU y ubicación.

Invariantes Disponible = física − comprometida; no negativo; tránsito no es stock utilizable en destino antes de recibir.

Eventos ReservaRegistrada/ReservaLiberada como candidatos; prototipo usa eventos de Pedido que describen la operación conjunta.

Estado del código Implementado reserva/liberación; movimientos por transferencia diseñados.

CuentaCredito

Raíz y contenido CuentaCredito; condiciones y decisión excepcional

Responsabilidades Evaluar condición normal y registrar excepción autorizada.

Invariantes Cliente empresarial activo, sin bloqueo y con crédito suficiente; excepción requiere responsable y motivo.

Eventos CreditoEvaluado, ExcepcionCreditoResuelta.

Estado del código Evaluación normal implementada como política de dominio; persistencia y autorización excepcional pendientes.

Transferencia

Raíz y contenido Transferencia; partidas y estados

Responsabilidades Solicitar, autorizar/rechazar, despachar y recibir.

Invariantes Origen, destino, responsable y estado obligatorios; recepción actualiza tránsito y destino con trazabilidad.

Eventos TransferenciaSolicitada, Despachada, Recibida o ConIncidencia.

Estado del código Diseño de dominio; fuera del código detallado U3.

Entrega

Raíz y contenido Entrega; dirección, asignación y evidencia

Responsabilidades Programar, registrar resultado e incidencia.

Invariantes Salida requiere datos completos; cierre conserva receptor, fecha/hora y evidencia; incidencia no equivale a entregado.

Eventos EntregaProgramada, EntregaCompletada, IncidenciaEntregaRegistrada.

Estado del código Diseño de dominio; sin optimización de rutas.

Devolucion

Raíz y contenido Devolucion; partidas y evaluación

Responsabilidades Evaluar elegibilidad y registrar destino financiero y físico.

Invariantes Comprobante, categoría, condición y política configurable; no inventar plazos o porcentajes.

Eventos DevolucionAutorizada o Rechazada.

Estado del código Diseño de dominio; política exacta pendiente de FerreMax según BRD.

Los siete límites evitan que FerreMax se convierta en un único objeto gigante. Cada raíz protege sus propios cambios; las referencias a otras raíces se conservan por identidad. La transacción de confirmación coordina Pedido e InventarioUbicacion, pero no elimina sus límites conceptuales.

Producto, Categoria, Ubicacion y ClienteEmpresarial son conceptos con identidad del catálogo o datos maestros. SKU identifica el producto; no se convierte automáticamente en una entidad adicional. Las decisiones sobre repositorios de maestros quedan fuera del código detallado.

Value Objects y entidades

Value Objects: SKU, Dinero, Vigencia y, en el diseño general, DireccionEntrega y evidencia estructurada del resultado. No tienen una identidad de negocio independiente de sus valores. Dinero conserva importe y moneda; Vigencia conserva emisión y vencimiento.

Entidades: las raíces anteriores, ClienteEmpresarial, Producto, Ubicacion, PartidaPedido y Reserva. Una partida se identifica dentro de su pedido y puede conservar cantidades solicitadas, surtidas y pendientes. El código mínimo usa un registro Partida inmutable con número local, SKU, cantidad y

precio; los estados por partida del proceso completo quedan modelados y pendientes de implementación.

Domain Services: PoliticaCredito para evaluación normal y PoliticaSurtidoMultiorigen para priorizar ubicación local, almacén central y otras sucursales según BR07. PoliticaDevolucion evalúa categoría y condición cuando FerreMax defina sus parámetros. La última y la de surtido son propuestas de diseño, no clases ya ejecutables.

Glosario actualizado

Término	Definición y responsabilidad
InventarioUbicacion	Estado del inventario de un SKU en una ubicación; raíz que controla sus reservas.
Reserva	Cantidad comprometida para un pedido confirmado; no equivale a cotización ni a existencia en tránsito.
Invariante	Condición del negocio que debe quedar verdadera al finalizar una operación válida.
Aggregate y raíz	Límite de consistencia y objeto por el que se solicitan sus modificaciones.
Evento de dominio	Hecho ocurrido y confirmado; por ejemplo PedidoConfirmado. No es una orden para confirmar.
Application Service	Coordina un caso de uso y solicita acciones al dominio; no decide todas las reglas.
Repository	Contrato para recuperar o guardar objetos del negocio; su implementación contiene SQL.
Unit of Work	Límite que confirma juntos los cambios de la operación o los revierte ante fallo.
Command	Intención de cambiar estado, con datos suficientes para la operación.
Query y DTO	Solicitud de solo lectura y estructura de respuesta apropiada para la consulta.
Outbox	Registro técnico del evento en la misma transacción; su publicación externa requiere un consumidor posterior.
ADR	Registro de una decisión, su contexto, alternativas, consecuencias y trazabilidad.

4 Service Layer

Servicio	Método	Entrada	Colaboradores	Salida	Reglas delegadas
CotizacionService	revalidar	Actor, ID, versión, vigencia, partidas, condiciones y motivo	UnitOfWork; CotizacionRepository; InventarioRepository; Cotizacion	Nueva versión y folio	BR01-03; validación de partidas y vigencia.
CotizacionService	convertir	Actor, ID, versión y aceptación	CotizacionRepository; InventarioRepository; PedidoRepository; Cotizacion	ID, estado, folio/versión y disponibilidad consultada	Vigencia y aceptación en Cotizacion; no reserva.
PedidoService	confirmar	Actor Caja, pedido, AutorizacionPago y ubicación	PedidoRepository; InventarioRepository; UnitOfWork; Pedido; InventarioUbicacion	ID, estado, referencia y ubicación	Pago y transición en Pedido; disponibilidad y reserva en InventarioUbicacion.
PedidoService	cancelar	Actor con permiso, pedido y motivo	PedidoRepository; InventarioRepository; UnitOfWork; Pedido	ID y estado	Estado y surtido en Pedido; liberación en InventarioUbicacion.
Handlers de consulta	handle	Actor y SKU o ID de pedido	DisponibilidadReader o PedidoReader	DTO independiente	Autorización de acceso; consultas de solo lectura.

El control de rol pertenece a la frontera de aplicación. El demo recibe un Actor suministrado por la composición de la prueba; todavía no verifica sesiones reales de usuarios. Los nombres de perfiles del prototipo ilustran permisos separados y deben traducirse a permisos nominales definidos por FerreMax, conforme a las restricciones del BRD.

El reloj y la fábrica de UnitOfWork se inyectan. Así, la vigencia se puede probar de forma determinista y la aplicación puede trabajar con memoria o PostgreSQL sin trasladar SQL al dominio.

5 Persistencia y transacciones

Contrato	Necesidad	Operaciones	Implementación
CotizacionRepository	Recuperar una versión exacta y conservar otra nueva	obtener; guardar_nueva_version	Tabla cotizacion_version; clave ID/versión; rechaza versión duplicada.
PedidoRepository	Recuperar y guardar el pedido	obtener; guardar	Tabla pedido; lectura para actualización con bloqueo de la fila.
InventarioRepository	Recuperar inventario consistente y sus reservas	obtener_para_actualizar; guardar; ubicacion_reserva	Inventario por SKU/ubicación y reservas; bloqueo del padre y actualización conjunta.
DisponibilidadReader	Información global sin materializar agregados	consultar_disponibilidad	SELECT parametrizado; devuelve cuatro cantidades por ubicación.
PedidoReader	Resumen apropiado para lectura	consultar_pedido	Proyección de ID, estado, cotización y total.

Los contratos se definen en application/ports.py y hablan en términos del dominio. Su ubicación es un límite de dependencias de la aplicación; el modelo de negocio no importa las implementaciones. PostgreSQL y Memory cumplen los contratos; el ensayo real debe verificar esa sustitución.

Transacción de confirmación

1. La aplicación comprueba el rol y abre UnitOfWork.

2. Recupera el pedido para actualización.

3. Recupera las filas de inventario involucradas con FOR UPDATE en un orden uniforme por SKU.

4. Pedido valida el pago y cambia provisionalmente a Confirmado.

5. Cada InventarioUbicacion verifica disponible y registra la reserva.

6. Guarda pedido y reservas, auditoría y evento en outbox.

7. Hace COMMIT; solo entonces devuelve el resultado confirmado.

8. Si cualquiera de esos pasos falla, hace ROLLBACK y no devuelve confirmación.

La fila de inventario bloqueada evita que dos transacciones decidan sobre la misma última unidad a partir de valores incompatibles. Los bloqueos se toman siempre en orden consistente; no se prometen objetivos de rendimiento sin medirlos. La lectura de disponibilidad es informativa y puede cambiar antes de la confirmación.

La cancelación aplica la misma unidad transaccional para cambiar el estado y liberar todas las reservas incluidas. La revalidación agrega una versión y su auditoría sin reservar stock. Cada consulta de lectura usa una transacción de solo lectura.

Eventos y auditoría

La outbox conserva el evento dentro de la transacción del negocio; un fallo no deja un evento de confirmación persistido para una operación revertida. El código no incluye aún un consumidor externo: no afirma haber enviado notificaciones.

La auditoría guarda actor, momento, acción, valores anterior/nuevo y motivo. La aplicación dispone de INSERT y SELECT, pero no UPDATE ni DELETE sobre esa tabla. Esto protege frente al usuario operativo; no pretende impedir la administración legítima de la base por su propietario.

PostgreSQL en Supabase

La decisión tecnológica conserva las responsabilidades anteriores. Supabase aloja PostgreSQL y aporta administración del proyecto. La aplicación Python utiliza una conexión PostgreSQL para coordinar las transacciones de UnitOfWork.

Se propone el esquema privado ferremax_u3, con cotizacion_version, pedido, inventario, reserva, auditoria y outbox. Las claves y relaciones reflejan el prototipo; los agregados no se deducen mecánicamente de esas seis tablas. Las versiones se insertan y se conservan; las reservas referencian pedido e inventario.

El esquema no se expone a anon ni authenticated de la Data API. Se habilita RLS y se concede acceso al grupo de backend confiable. La conexión de aplicación debe usar un usuario con ese rol y privilegios limitados; la conexión administrativa se utiliza para preparar el esquema. La contraseña nunca pertenece al repositorio.

El adaptador PostgreSQL y el esquema están preparados; su prueba con conexiones reales sigue pendiente.

6 Commands y Queries

Tipo	Operación	Handler	Dependencias	Resultado
Command	ConfirmarPedido	ConfirmarPedidoHandler	PedidoService; UoW; repositorios de pedido e inventario	Confirmación y reserva, o error de negocio.
Command	CancelarPedido	CancelarPedidoHandler	PedidoService; UoW; repositorios de pedido e inventario	Cancelación y liberación, o rechazo.
Query	ConsultarDisponibilidad	ConsultarDisponibilidadHandler	DisponibilidadReader	SKU, encontrado y lista de ubicaciones con cantidades.
Query	ConsultarPedido	ConsultarPedidoHandler	PedidoReader	ID, estado, folio/versión de cotización, total y moneda.

Se conserva una sola base PostgreSQL. Los DTO son estructuras de lectura y no se usan para modificar automáticamente entidades. Los comandos delegan al servicio para conservar un único recorrido de reglas y transacción; no duplican la lógica en el handler.

La emisión y revalidación pueden exponerse inicialmente como métodos del servicio. No es necesario añadir handlers idénticos solo para incrementar clases. La separación obligatoria queda demostrada con los dos comandos y las dos consultas seleccionados.

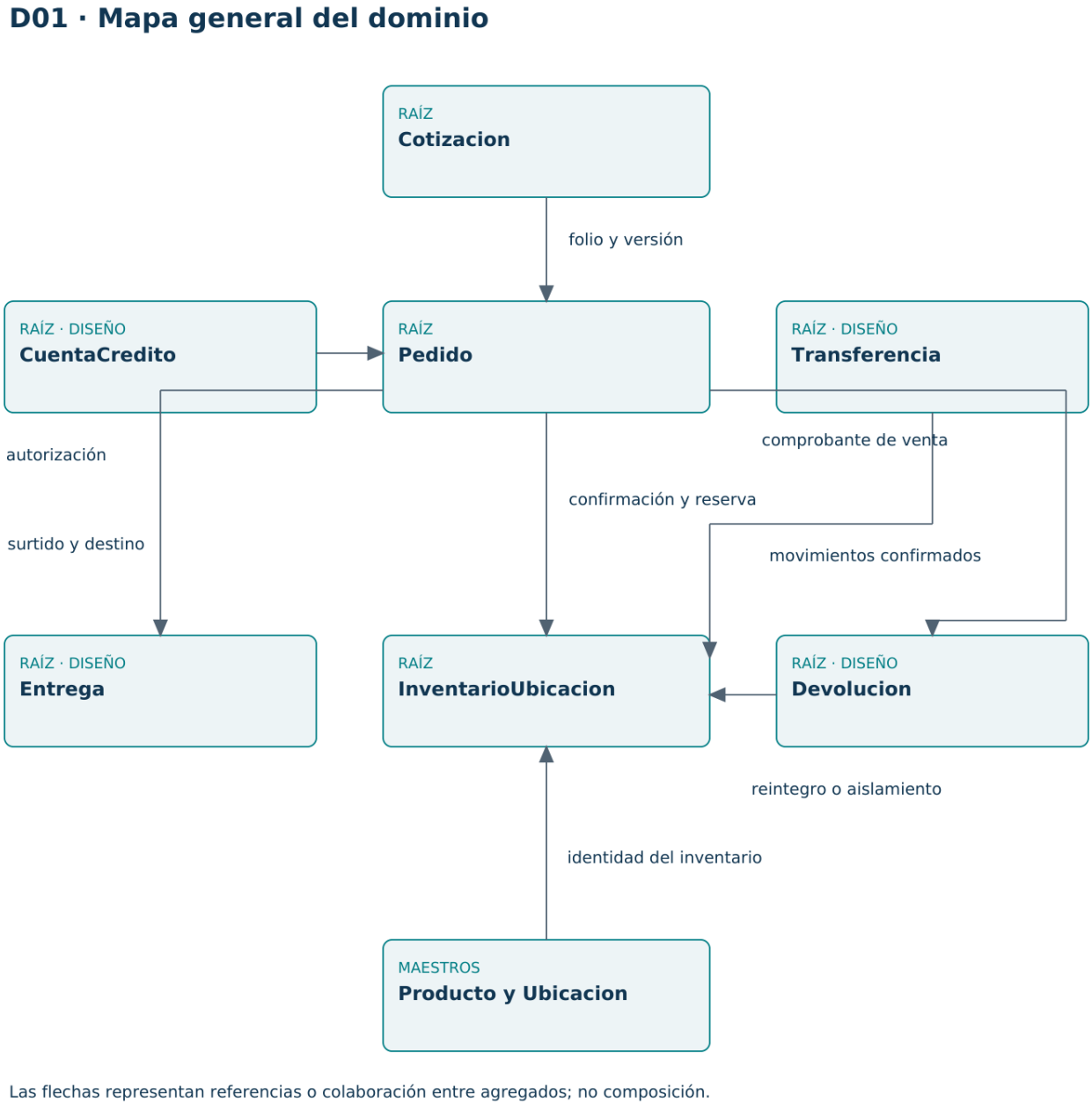
7 Revisión con SOLID

Principio	Evidencia	Impacto	Cambio o control
S	Las reglas de vigencia y reserva están en Cotizacion e InventarioUbicacion; el servicio coordina.	Mezclarlas con SQL haría cambiar el servicio por políticas y por almacenamiento.	Separar model.py, services.py y postgres.py; mover ubicacion_reserva al repositorio de inventario.
O	PoliticaCredito expresa evaluación normal; devolución y surtido tienen variaciones pendientes.	Una cadena creciente de condiciones mezclaría políticas distintas.	Mantener políticas separadas; añadir implementaciones solo cuando haya variantes justificadas, sin inventar categorías.
L	Repositorios Memory y PostgreSQL ofrecen operaciones equivalentes y rechazos de negocio.	Una implementación que pierde versiones o permite reservas duplicadas rompería el contrato.	Conservar versiones, rechazar SKU inexistente y aplicar rollback. Prueba de contrato PostgreSQL pendiente de conexión.
I	DisponibilidadReader y PedidoReader no obligan a una consulta a guardar o eliminar.	Una interfaz universal expondría métodos irrelevantes al lado lector.	Contratos pequeños para consultas y repositorios especializados por agregado.
D	Servicios reciben una fábrica UnitOfWork y un reloj; el dominio no importa SQL ni Psycopg.	Dependencia directa del backend impediría revisar el dominio sin una conexión.	Adaptadores PostgreSQL y Memory; consultas parametrizadas y composición externa de dependencias.

8 Representación de la arquitectura

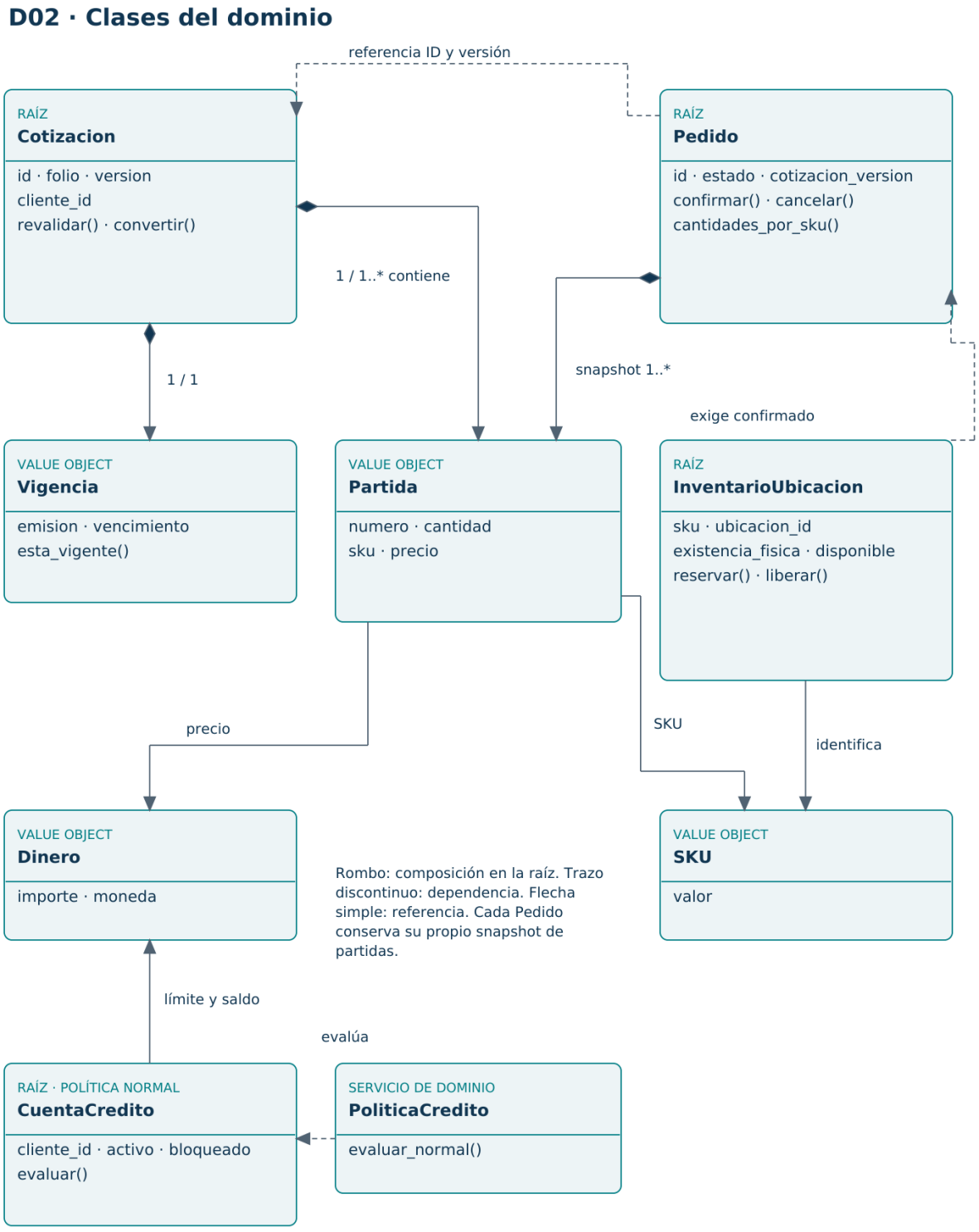
D01 presenta el mapa general. D02 describe las clases del dominio y D03 las de aplicación e infraestructura. D04 recorre la confirmación con reserva y D05 muestra el rechazo por falta de disponibilidad. Los nombres corresponden con los componentes del código.

D01 Mapa general del dominio



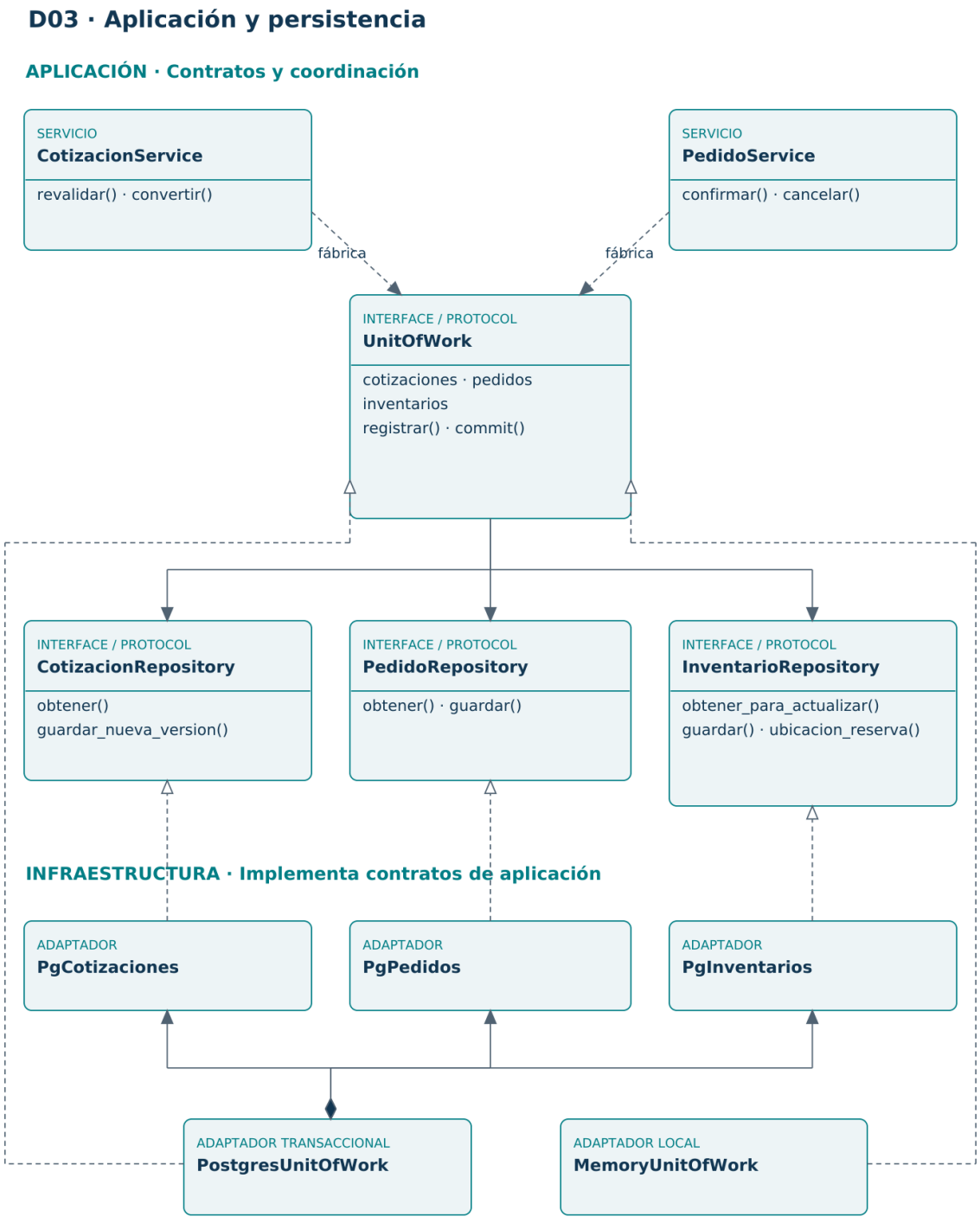
Fuente D01.mmd y exportación D01.pdf en diagrams.

D02 Clases del dominio



Fuente D02.mmd y exportación D02.pdf en diagrams.

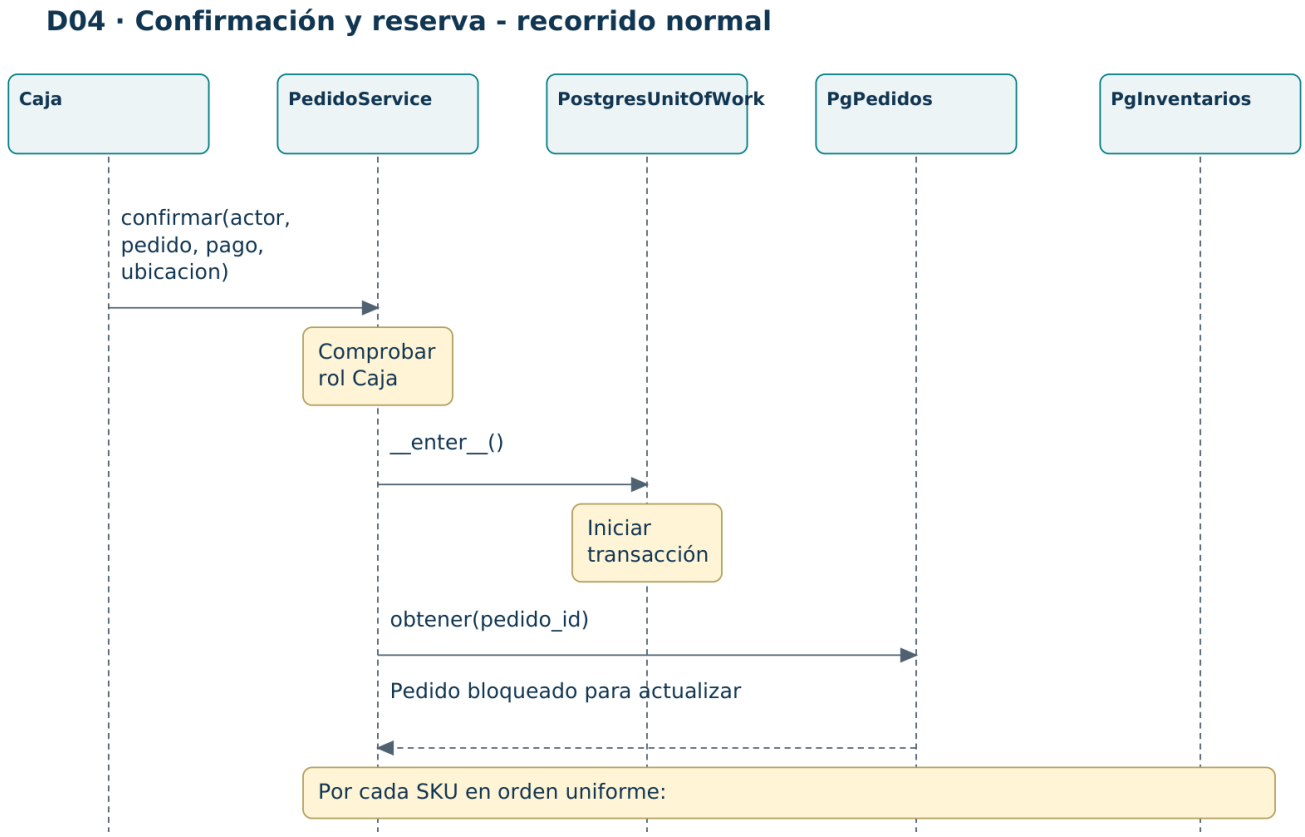
D03 Aplicación e infraestructura



Triángulo vacío y trazo discontinuo: implementación del contrato. Rombo: adaptadores contenidos por el UoW. El dominio se detalla en D02.

Fuente D03.mmd y exportación D03.pdf en diagrams.

D04 Confirmación y reserva

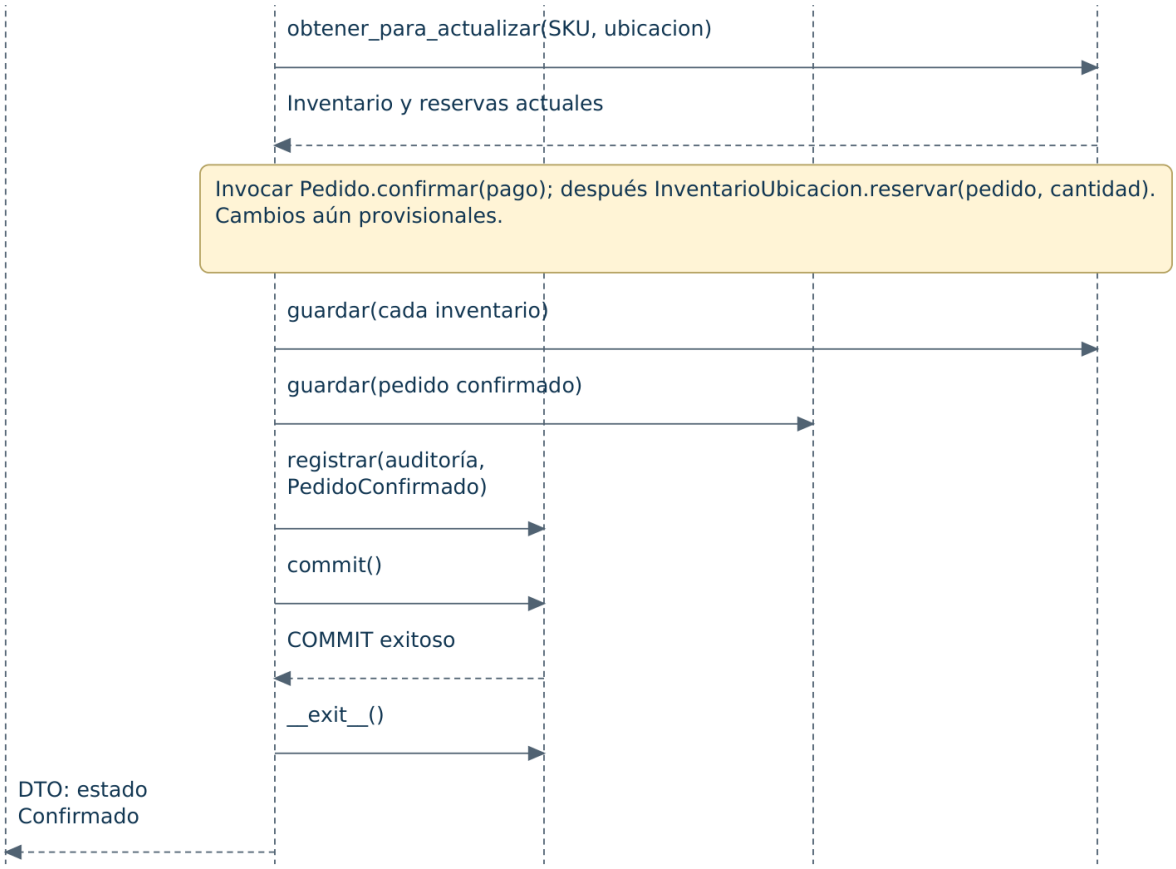


Fuente D04.mmd y exportación D04.pdf en diagrams.

El servicio valida el rol, obtiene el pedido y bloquea el inventario por SKU en un orden uniforme. Pedido e InventarioUbicacion aplican sus reglas; la confirmación, las reservas, la auditoría y el evento se guardan juntos. La respuesta Confirmado se emite después del COMMIT.

D04 Confirmación y reserva continuación

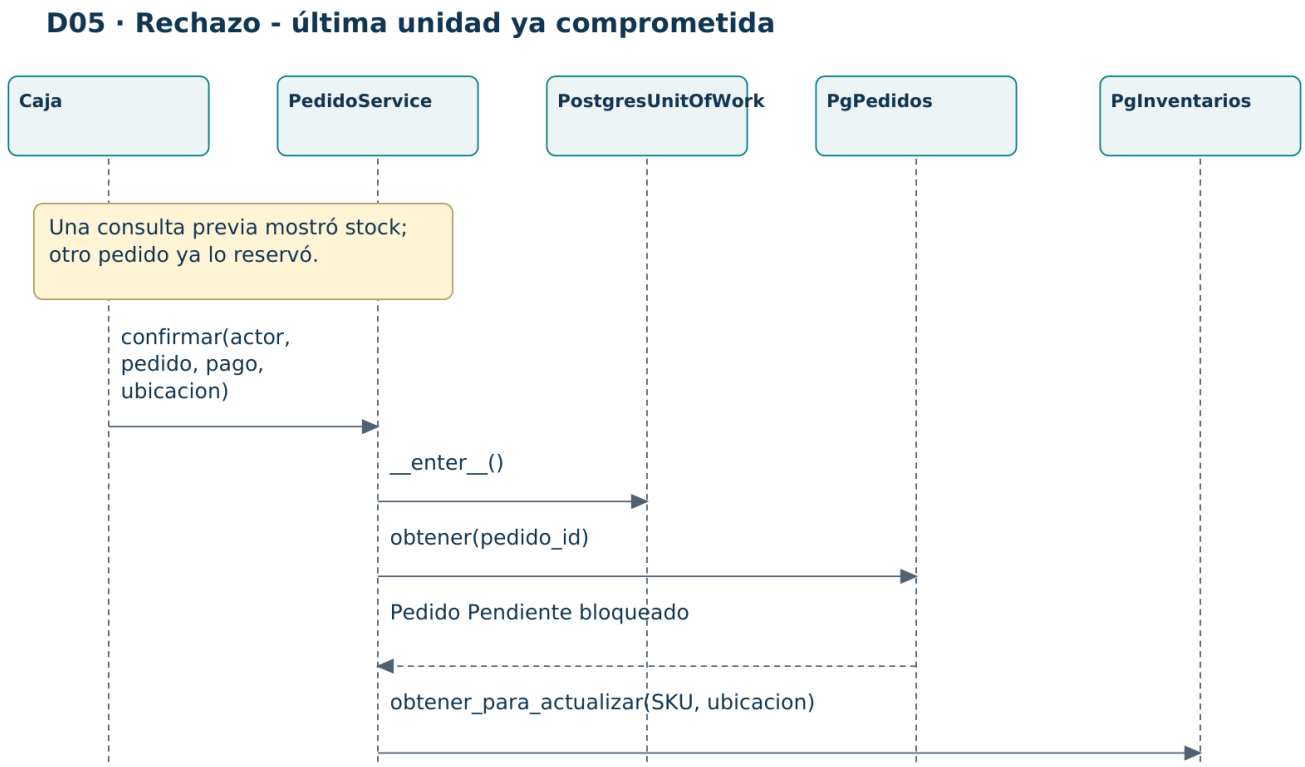
Participantes de izquierda a derecha: Caja, PedidoService, PostgresUnitOfWork, PgPedidos y PgInventarios.



Fuente D04.mmd y exportación D04.pdf en diagrams.

La outbox conserva PedidoConfirmado dentro de la misma transacción. Su registro acredita el hecho del negocio; el consumidor externo que lo publicará es una ampliación posterior.

D05 Rechazo por disponibilidad insuficiente

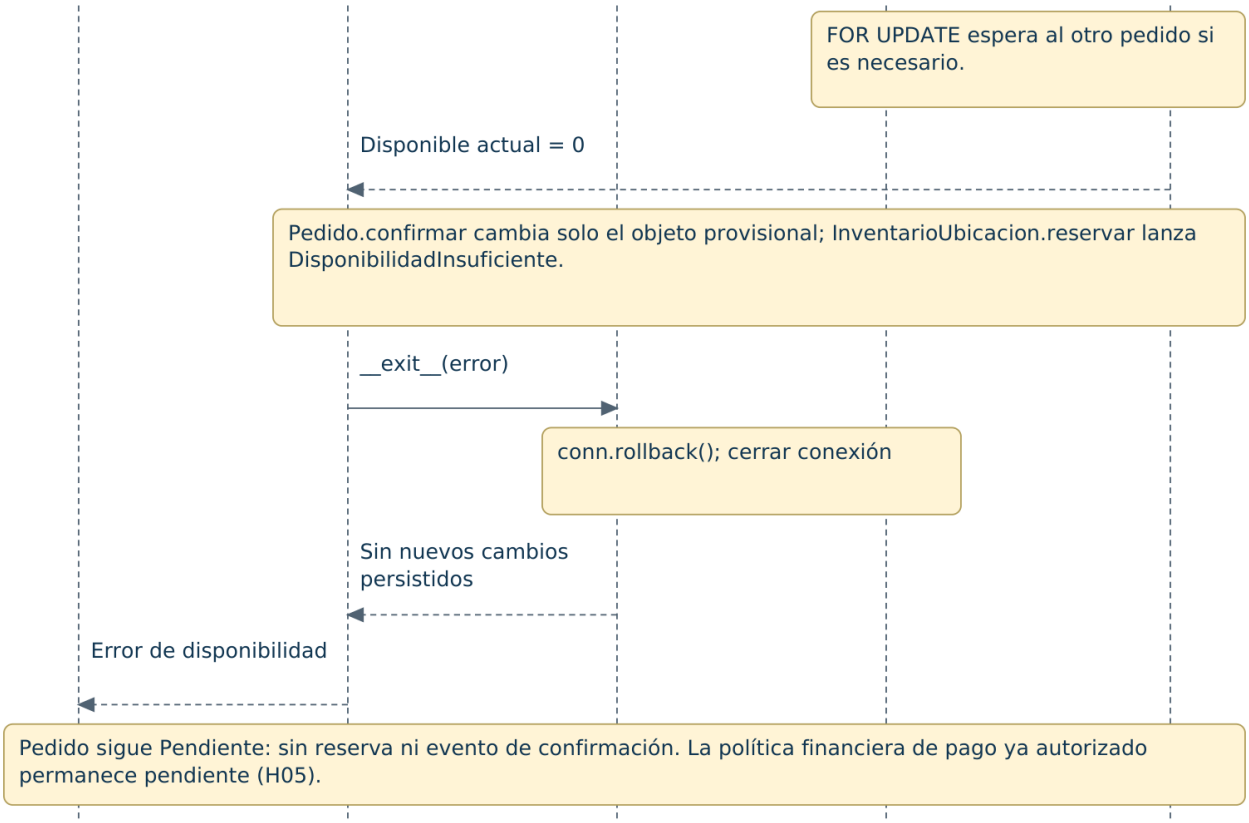


Fuente D05.mmd y exportación D05.pdf en diagrams.

La consulta previa de disponibilidad es informativa. Al confirmar se recupera el inventario actual con bloqueo; si otro pedido consumió la última unidad, la operación detecta disponibilidad insuficiente.

D05 Rechazo por disponibilidad insuficiente continuación

Participantes de izquierda a derecha: Caja, PedidoService, PostgresUnitOfWork, PgPedidos y PgInventarios.



Fuente D05.mmd y exportación D05.pdf en diagrams.

El rollback descarta la confirmación y todas las reservas provisionales. El pedido permanece Pendiente. El tratamiento financiero de un pago autorizado sin reserva sigue siendo la política pendiente H05; no se presupone un reembolso automático.

Registros de decisiones arquitecturales

ADR001 Domain Model como patrón principal

Estado: Propuesta para revisión del equipo.

Contexto

Las cotizaciones conservan versiones y vigencia; el pedido cambia de estado; inventario y reservas deben mantenerse consistentes. Esas reglas reaparecen en varios casos de uso.

Alternativas

Transaction Script ofrece menor estructura inicial, con riesgo de duplicar reglas. Table Module organiza conjuntos de datos, pero coloca el ciclo de vida del pedido fuera de los objetos que representan el negocio. Domain Model encapsula reglas y conserva identidad y comportamiento.

Decisión

Proponer Domain Model en la escritura y consultas SQL con DTO propios en la lectura. No clasificar cualquier SELECT como Table Module ni introducir una combinación de patrones sin delimitarla.

Consecuencias

Beneficio: responsabilidades visibles y reglas reutilizables. Costo: más objetos, contratos y validación de límites. Riesgo: clases anémicas o servicios que absorban el dominio. Control: métodos de negocio y cinco hallazgos SOLID.

Trazabilidad

BN02/BN03; BR01-06; RNF02; US04/US05/US09/US10; CU02-CU05.

ADR002 Python y PostgreSQL alojado en Supabase

Estado: Tecnología y organización confirmadas por Roberto; proyecto aún no creado.

Contexto

El profesor permite elegir tecnología. Roberto propuso PostgreSQL en Supabase y autorizó continuar con esa opción; eligió Santana’s Org para un proyecto nuevo.

Alternativas

SQLite simplifica una demostración local. PostgreSQL local conserva las funciones del motor pero requiere instalación/operación. Supabase aloja PostgreSQL y permite administración y persistencia compartida.

Decisión

Python organiza el dominio y la aplicación. Un adaptador PostgreSQL conserva transacciones y repositorios. Supabase se utiliza inicialmente para alojar la base; se conserva una única base de lectura y escritura.

Consecuencias

Beneficio: práctica con persistencia compartida y transacciones de PostgreSQL. Costo: conexión, permisos, dependencias y aprovisionamiento. El costo de creación debe verificarse antes de activar un proyecto; no se asume que una organización sea gratuita. No hay proyecto activo de FerreMax ni pruebas conectadas reportadas.

Trazabilidad

RNF02/03/04/08; BR04/05/16; CU04/CU05; conversación de decisión tecnológica y Release.

ADR003 Unit of Work para confirmación y reserva

Estado: Propuesta implementada y revisada con Memory; validación PostgreSQL pendiente.

Contexto

AC-US09-01 exige reserva atómica y AC-US09-02 impide asignar la última unidad a dos pedidos. Guardar el estado del pedido separado de la reserva puede producir confirmaciones incoherentes.

Alternativas

Guardar por separado introduce cambios parciales. Coordinar con consistencia eventual añade recuperación y estados de compensación aún no definidos. Una transacción local puede proteger los cambios requeridos en una misma base.

Decisión

El servicio abre UoW, bloquea pedido e inventarios en orden, invoca al dominio, guarda pedido/reservas/auditoría/outbox y confirma una sola transacción. Ante error, revierte la operación completa.

Consecuencias

Beneficio: confirmación y compromiso de inventario quedan juntos. Costo: retención breve de bloqueos y manejo de contención. La transacción abarca más de un agregado sin convertirlos en una sola raíz. Las conexiones y los bloqueos reales necesitan ensayo PostgreSQL; RNF01 no se declara cumplido sin medirlo.

Trazabilidad

BN02; BR04-06/BR16; RNF02/RNF04; AC-US06-01/02; AC-US09-01/02; CU04/CU05.

ADR004 CQRS básico sobre una sola base

Estado: Propuesta implementada en comandos y consultas seleccionados.

Contexto

La consulta global debe distinguir física, comprometida, tránsito y disponible. Confirmar y cancelar cambian estado y exigen validaciones diferentes a una consulta.

Alternativas

Un modelo único para pantallas y escritura simplifica estructuras pero puede permitir modificaciones involuntarias. Bases separadas exigirían sincronización y operación adicional. Separar handlers y modelos conservando PostgreSQL resuelve la necesidad actual.

Decisión

ConfirmarPedido y CancelarPedido son Commands. ConsultarDisponibilidad y ConsultarPedido son Queries con contratos de lectura y DTO propios. Los commands delegan a la Service Layer; una consulta no recibe métodos para guardar.

Consecuencias

Beneficio: intención explícita y lectores pequeños. Costo: DTO y handlers adicionales. No implica Event Sourcing, réplicas ni dos bases. La separación se aplica donde aporta claridad y no obliga a crear un handler decorativo por cada método.

Trazabilidad

BN01/BN02; BR04-06; RNF01/RNF06; US01/US02/US09/US10; CU01/CU04/CU05.

ADR005 Auditoría y outbox junto con el cambio de negocio

Estado: Propuesta implementada como registro; consumidor externo pendiente.

Contexto

BR16 y RNF04 exigen quién, cuándo, qué cambió y por qué. Publicar un evento antes del COMMIT puede comunicar una confirmación que posteriormente falla.

Alternativas

Auditar/publicar por separado puede perder registros o emitir hechos no confirmados. Añadir un intermediario de mensajes externo aumenta trabajo sin un consumidor definido. Guardar auditoría y evento junto con el cambio permite conservar evidencia coherente.

Decisión

Registrar auditoría y evento en la misma transacción. El backend operativo tiene permiso de insertar y leer auditoría, sin actualizarla ni borrarla. Una outbox técnica conserva eventos pendientes para un consumidor posterior.

Consecuencias

Beneficio: coherencia entre cambio y evidencia. Costo: tabla adicional y necesidad futura de publicación/idempotencia. El prototipo registra eventos, pero no afirma entregar notificaciones ni garantizar exactamente una entrega externa.

Trazabilidad

BN10; BR16; RNF04; CA-US20-01; CU02-CU05.

9 Validación y trazabilidad

El núcleo final ejecutó 20 pruebas locales correctamente: rechazo de cotización vencida; conservación de versiones; aceptación obligatoria; confirmación con reserva, auditoría y evento; pago rechazado; reversión de todas las partidas al fallar una; acumulación de partidas con el mismo SKU; dos pedidos para una última unidad; liberación por cancelación; rechazo de cancelación después del surtido; consulta sin cambios; control de rol; evaluación de crédito; distinción entre tránsito y disponible; valores inválidos.

La prueba de competencia en Memory usa un bloqueo de proceso. Se corrigió el fixture a ZZ-SIN-STOCK y se agregó una observación que confirma el guardado provisional de SKU-DEMO antes del fallo. Luego se comprueba la reversión completa. Las 20 pruebas locales pasaron; una prueba PostgreSQL fue omitida por falta de TEST_DATABASE_URL. Falta ejecutar la prueba PostgreSQL con conexiones separadas para verificar los bloqueos de la base y la sustitución del adaptador.

Control	Resultado
Trazabilidad de los cinco casos	Relacionados con BN, BR, US y escenarios existentes.
Lenguaje	Términos del glosario conservados; vocabulario técnico añadido con definición.
Responsabilidad	Reglas en dominio; coordinación en servicios; SQL en infraestructura.
Atomicidad local	Probada la reversión del pedido y de una primera reserva guardada provisionalmente antes del fallo de otra partida.
Persistencia PostgreSQL	Adaptador preparado; prueba conectada pendiente.
Diagramas	Cinco fuentes Mermaid y vistas vectoriales SVG, PDF y PNG. BPMN de U2 verificado; hallazgo H07 documentado.
IA	Iteraciones y correcciones registradas; revisión técnica IA diferenciada de revisión humana.
Rendimiento RNF01	Objetivos conservados; no medidos ni declarados cumplidos.

Matriz de trazabilidad por necesidad

ID	Reglas y RNF	Historias	Caso o alcance	Componentes	ADR	Cobertura
BN01	BR04; RNF01	US01/US02	CU01	ConsultarDisponibilidadHandler; DisponibilidadReader	ADR004	Consulta implementada; objetivo temporal sin medir.

ID	Reglas y RNF	Historias	Caso o alcance	Componentes	ADR	Cobertura
BN02	BR04-06; RNF02	US09/US10	CU04/CU05	Pedido; InventarioUbicacion; UnitOfWork	ADR001/003	Reserva y liberación de contado en una ubicación.
BN03	BR01-03; RNF08	US03-05	CU02/CU03	Cotizacion; CotizacionService; CotizacionRepository	ADR001	Revalidación y conversión implementadas; emisión inicial simulada.
BN04	BR07/08	US05/ US11/ US14/US15	CU04 más diseño general	Pedido; PoliticaSurtidoMultiorigen; Transferencia	ADR001/003	Multiorigen y surtido diseñados; prototipo local.
BN05	BR09-11	US07/US08	Diseño comercial	CuentaCredito; PoliticaCredito	ADR001	Evaluación normal probada; excepciones y persistencia pendientes.
BN06	BR12/13	US11-13	Diseño logístico	Transferencia; InventarioUbicacion	ADR001/003	Mapeado; sin flujo ejecutable de transferencia.
BN07	BR08	US14	Diseño comercial	Pedido; PartidaPedido; aceptación del cliente	ADR001	Mapeado; sin aplicación ejecutable de parciales.
BN08	BR14; RNF07	US16/US17	Diseño logístico	Entrega; DireccionEntrega; EvidenciaEntrega	ADR001	Mapeado; sin programación de unidad ni captura operativa.
BN09	BR15; RNF08	US18	Diseño de postventa	Devolucion; PoliticaDevolucion; Categoria	ADR001	Política exacta pendiente del negocio.
BN10	BR16; RNF04	US19/US20	CU02-CU05 y diseño de panel	Auditoria; Outbox; DTO de reportes	ADR005	Auditoría local implementada; panel completo pendiente.

Matriz de trazabilidad por regla

Regla	Escenarios U2	Caso o alcance	Componente responsable	Evidencia o límite
BR01	US03/US04/ US05	CU02/CU03	CotizacionService	No llama reservar durante revalidación/conversión; prueba sin stock comprometido.
BR02	US03/US04	CU02/CU03	Cotizacion; CotizacionRepository	Folio, versión y vigencia conservados; estado vigente se deriva de Vigencia.
BR03	AC-US04-01/02	CU02/CU03	Cotizacion.convertir/ revalidar	Vencida rechazada; versión anterior conservada.
BR04	US02; AC-US09-01	CU01/CU04	InventarioUbicacion.disponible	Física menos reservas; tránsito informado por separado.
BR05	US06; AC-US09-01	CU04	Pedido.confirmar; InventarioUbicacion.reservar	Dominio exige confirmado; UoW guarda estado y reserva juntos.
BR06	AC-US10-01	CU05	Pedido.cancelar; InventarioUbicacion.liberar	Cancelación antes de surtido libera; reducción es variante futura.
BR07	US01/US11	Diseño general	PoliticaSurtidoMultiorigen	Prioridad local, almacén, otras sucursales; prototipo no implementa esta selección.
BR08	AC-US14-01/02	Diseño general	Pedido y PartidaPedido	Aceptación explícita para parciales; no usar excepción de stock como consentimiento.
BR09	AC-US07-01	Diseño general	CuentaCredito.evaluar	Activo es condición de autorización normal.
BR10	AC-US07-01/02	Diseño general	PoliticaCredito	Bloqueo o insuficiencia devuelven EnRevision.
BR11	AC-US08-01/02	Diseño general	CuentaCredito y decisión excepcional	Motivo, responsable, fecha y condición requeridos; no inventar umbrales de excepción.
BR12	AC-US11-01; US12	Diseño logístico	Transferencia	Origen, destino, partidas, responsable y estado.
BR13	AC-US13-01/02	Diseño logístico	Transferencia e InventarioUbicacion	Recepción actualiza tránsito/destino; diferencia se conserva como incidencia.

Regla	Escenarios U2	Caso o alcance	Componente responsable	Evidencia o límite
BR14	AC-US17-01/02	Diseño logístico	Entrega	Evidencia de receptor y resultado; incidencia no cierra como entregado.
BR15	CA-US18-01	Diseño postventa	Devolucion y PoliticaDevolucion	Evaluación por categoría/condición y política pendiente.
BR16	CA-US20-01	CU02-CU05	UnitOfWork.registrar; auditoria	Valores anteriores/nuevos, actor, fecha y motivo dentro de la transacción.

Trazabilidad de los rechazos

Escenario	Entrada o situación	Resultado y comprobación
AC-US04-01	Cotización fuera de vigencia	CotizacionVencida; no crea pedido.
AC-US06-02	Pago rechazado	Pedido sigue Pendiente; sin reserva ni evento de confirmación.
AC-US09-02	Dos pedidos intentan la última unidad	Uno confirma; otro recibe DisponibilidadInsuficiente. Memory probado; PostgreSQL pendiente.
AC-US10-01	Cancelación autorizada antes del surtido	Estado Cancelado y stock disponible restituido.
AC-US01-02 propuesto	SKU inexistente	encontrado falso y lista vacía; ID original repetido registrado en H01.

Evidencia de ejecución local

El ensayo ejecutó 21 pruebas: 20 correctas y una integración PostgreSQL omitida por falta de TEST_DATABASE_URL. Los registros completos están en docs/evidence. Memory protege la competencia con un bloqueo de proceso; el resultado no certifica los bloqueos ni el rendimiento de PostgreSQL.

```
----------------------------------------------------------------------
Ran 21 tests in 0.014s

OK (skipped=1)
```

10 Uso de IA

La IA apoyó la comparación de alternativas, el código y la documentación. La bitácora distingue las decisiones expresamente confirmadas por Roberto de las propuestas técnicas. La revisión conjunta del equipo y la defensa no se registran como realizadas.

Iteración	Objetivo	Prompt o interacción real	Respuesta útil	Error o supuesto detectado	Decisión y autor
IA01	Entender la guía	Solicitud real de Roberto: revisar Word, captura y cuatro PDFs; comunicar dudas antes de resolver.	Resumen de nueve etapas y mínimos; propuesta de proceso.	BPMN visual no disponible; ID repetido y OB01/CE01 incongruentes.	Roberto informó que no hay repo, hay libertad tecnológica y revisión el viernes. No implica aprobación de cada clase.
IA02	Elegir persistencia	Propuesta inicial Python/SQLite; pregunta real de Roberto sobre Supabase PostgreSQL.	Comparación de preparación, trabajo compartido y concurrencia; propuesta Python/PostgreSQL.	SQLite se había propuesto por simplicidad; no se había provisionado una base.	Roberto dio Release tras la propuesta Supabase y después eligió Santana's Org.
IA03	Revisar límites del código	Revisión técnica interna de la implementación por IA; no se atribuye al equipo un prompt que no escribió.	Interfaces pequeñas, entidades con reglas y adaptadores.	ubicacion_reserva estaba en el UoW y debía estar en InventarioRepository.	IA corrigió contrato, adaptador Memory y llamada del servicio; revisión humana pendiente.
IA04	Revisar concurrencia y consistencia	Revisión técnica interna de revalidación y confirmación.	Orden uniforme por SKU y agregación de partidas del mismo producto.	Revalidación podía obtener bloqueos según orden de entrada, diferente al de confirmar.	IA ordenó lecturas y probó duplicados de SKU y competencia por última unidad.
IA05	Comprobar comportamiento	Ejecución real: PYTHONPATH=src python3 -m unittest discover -s tests -v.	15 pruebas locales correctas, incluyendo rollback y liberación.	La prueba Memory serializa con RLock; no certifica los bloqueos PostgreSQL.	Resultado conservado; integración PostgreSQL identificada por separado como pendiente.

Iteración	Objetivo	Prompt o interacción real	Respuesta útil	Error o supuesto detectado	Decisión y autor
IA06	Conservar evidencia ante interrupción	Continuación autorizada por Release; comprobación de acceso tras cierre de conexión local.	Documentación, diagramas y decisiones conservados en borrador privado de Notion.	Conexión Mac cerrada; get_cost no disponible; no se completaron PDF, GitHub ni Supabase.	IA continuó la documentación; no declaró creados recursos ni aprobada la arquitectura por ambos integrantes.

Correcciones posteriores

IA07: se fortaleció la prueba de dos partidas: faltante ZZ-SIN-STOCK y observación del guardado provisional de SKU-DEMO. El ensayo de rollback pasó.

IA08: se revisó visualmente BPMN y se registró H07 sobre reserva previa a autorización.

IA09: se reconstruyó el código en cloud, se agregó estado calculado de cotización y auditoría detallada. Pasaron 20 pruebas locales; se omitió una integración PostgreSQL por falta de conexión. El entorno se desconectó antes de confirmar generación de PDFs y empaquetado.

IA10 Se generaron y revisaron las exportaciones y el paquete del proyecto. **IA11** Se adaptó el informe al formato de la guía práctica, conservando el contenido de FerreMax y retirando instrucciones, rúbrica y ejemplos ajenos al proyecto.

11 Conclusiones

La arquitectura conecta las necesidades de FerreMax con componentes concretos: Cotizacion conserva vigencia y versiones; Pedido expresa confirmación y cancelación; InventarioUbicacion protege la disponibilidad. La Service Layer coordina esos componentes y el Unit of Work evita que una confirmación quede guardada sin las reservas correspondientes.

Domain Model se propone por la interacción entre reglas y estados, y acepta como costo una estructura mayor que Transaction Script. CQRS se mantiene básico, con una sola base y modelos de lectura propios. PostgreSQL en Supabase permite continuar el proyecto y revisar persistencia compartida, aunque añade configuración y requiere validar sus transacciones de forma real.

La prueba del diseño incluye también los rechazos. La cotización vencida, el pago rechazado y la disponibilidad insuficiente deben tener resultados comprensibles y conservar un estado coherente. Los procesos no implementados y las políticas pendientes se identifican expresamente para mantener la continuidad con U2.

12 Referencias

Gómez Marroquín, M. Á. (2026). Clases de Componentes de lógica de negocio de 9 y 14 de septiembre. Material del curso y apuntes corroborados en Notion.

Equipo 17. (2026). FerreMax: BRD, User Stories, Glosario y BPMN. Documentos de Unidad 2 corregidos.

Guía de trabajo para alumnos. (2026). Diseño arquitectural del sistema con IA. Actividad de Unidad 3.

Fowler, M. (s. f.). Patterns of Enterprise Application Architecture. Catálogo.

Supabase. (s. f.). Database y Connect to your database. Consulta: 2026-10-14.

PostgreSQL Global Development Group. (s. f.). Explicit locking.

Microsoft. (s. f.). CQRS pattern.